

Filtering Data

In this chapter, you will learn how to use the `SELECT` statement's `WHERE` clause to specify search conditions.

Using the WHERE Clause

Database tables usually contain large amounts of data, and you seldom need to retrieve all the rows in a table. More often than not, you'll want to extract a subset of a table's data as needed for specific operations or reports. Retrieving just the data you want involves specifying *search criteria*, also known as *filter conditions*.

Within a `SELECT` statement, you filter data by specifying search criteria in the `WHERE` clause. You include the `WHERE` clause right after the table name (the `FROM` clause) as follows:

► Input

```
SELECT prod_name, prod_price
FROM products
WHERE prod_price = 2.50;
```

► Analysis

This statement retrieves two columns from the `products` table, but instead of returning all rows, it returns only rows with the `prod_price` value 2.50, shown here:

► Output

+	-----+	-----	+	
	prod_name		prod_price	
+	-----+	-----	+	
	Carrots		2.50	
	TNT (1 stick)		2.50	
+	-----+	-----	+	

This example uses a simple equality test: It checks to see if a column has a specified value, and it filters the data accordingly. But SQL enables you to do more than just test for equality.

Tip

SQL Versus Application Filtering You can filter data at the application level. To do this, you use the SQL `SELECT` statement to retrieve more data than is actually required for the client application, and the client code loops through the returned data to extract just the needed rows.

As a rule, this practice is strongly discouraged. Databases are optimized to perform filtering quickly and efficiently. Making the client application (or development language) do the database's job dramatically impacts application performance and creates applications that cannot scale properly. In addition, if data is filtered at the client, the server has to send unneeded data across the network connections, resulting in wasted network bandwidth resources.

Caution

WHERE Clause Position When using both `ORDER BY` and `WHERE` clauses, make sure `ORDER BY` comes after `WHERE`; otherwise, you will get an error. (See Chapter 5, "Sorting Retrieved Data," for more information on using `ORDER BY`.)

WHERE Clause Operators

New Term

Operator A special keyword used to join or change clauses within a `WHERE` clause. Also known as a *logical operator*.

The first `WHERE` clause we looked at tests for equality to determine whether a column contains a specific value. MySQL supports a whole range of conditional operators, some of which are listed in Table 6.1.

Table 6.1 WHERE Clause Operators

Operator	Description
=	Equality
<>	Nonequality
!=	Nonequality
<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to
BETWEEN	Between two specified values

Checking Against a Single Value

We have already seen an example of testing for equality. Here's one more:

► Input

```
SELECT prod_name, prod_price
FROM products
WHERE prod_name = 'fuses';
```

► Output

prod_name	prod_price
Fuses	3.42

► Analysis

WHERE prod_name = 'fuses' returns a single row with the value Fuses. By default, MySQL is not case-sensitive when performing matches, and so fuses and Fuses match.

Now let's look at a few examples that demonstrate the use of other operators.

This example lists all products that cost less than 10:

► Input

```
SELECT prod_name, prod_price
FROM products
WHERE prod_price < 10;
```

► Output

prod_name	prod_price
.5 ton anvil	5.99
1 ton anvil	9.99
Carrots	2.50
Fuses	3.42
Oil can	8.99
Sling	4.49
TNT (1 stick)	2.50

This statement retrieves all products that cost 10 or less (resulting in two additional matches):

► Input

```
SELECT prod_name, prod_price
FROM products
WHERE prod_price <= 10;
```

► Output

prod_name	prod_price
.5 ton anvil	5.99
1 ton anvil	9.99
Bird seed	10.00
Carrots	2.50
Fuses	3.42
Oil can	8.99
Sling	4.49
TNT (1 stick)	2.50
TNT (5 sticks)	10.00

Checking for Nonmatches

This example lists all products not made by vendor 1003:

► Input

```
SELECT vend_id, prod_name
FROM products
WHERE vend_id <> 1003;
```

► Output

vend_id	prod_name
1001	.5 ton anvil
1001	1 ton anvil
1001	2 ton anvil
1002	Fuses
1005	JetPack 1000
1005	JetPack 2000
1002	Oil can

Tip

When to Use Quotes If you look closely at the conditions used in the WHERE clauses in the preceding examples, you will notice that some values are enclosed within single quotes (such as 'fuses'), and others are not. The single quotes are used to delimit strings. If you are comparing a value against a column that is a string datatype, the delimiting quotes are required. Quotes are not used to delimit values used with numeric columns.

The following example is the same as the previous one except that this one uses the `!=` operator instead of `<>`:

► Input

```
SELECT vend_id, prod_name
FROM products
WHERE vend_id != 1003;
```

Tip

Use `<>` and `!=` Interchangeably The operator `!=` means *not equal*, so `vend_id != 1003` means “match all vendors where `vend_id` is *not* 1003.” The operator `<>` means *less than or greater than*, so `vend_id <> 1003` means “match all vendors where `vend_id` is less than or greater than 1003 (but not the same).” As you can see, these two operators effectively do the same thing; you can use whichever one you prefer.

Checking for a Range of Values

To check for a range of values, you can use the `BETWEEN` operator. Its syntax is a little different from that of other `WHERE` clause operators because it requires two values: the beginning of the range and the end of the range. The `BETWEEN` operator can be used, for example, to check for all products that cost between 5 and 10 or for all dates that fall between specified start and end dates.

The following example demonstrates the use of the `BETWEEN` operator to retrieve all products with a price between 5 and 10:

► Input

```
SELECT prod_name, prod_price
FROM products
WHERE prod_price BETWEEN 5 AND 10;
```

► Output

prod_name	prod_price
.5 ton anvil	5.99
1 ton anvil	9.99
Bird seed	10.00
Oil can	8.99
TNT (5 sticks)	10.00

► Analysis

As you can see in this example, when **BETWEEN** is used, two values must be specified—the low end and the high end of the desired range. The two values must be separated by the **AND** keyword. **BETWEEN** matches all the values in the range, including the specified range start and end values.

Checking for No Value

When a table is created, the table designer can specify whether individual columns can contain no value. When a column contains no value, it is said to contain a **NULL** value.

New Term

NULL No value, as opposed to a field containing 0, or an empty string, or just spaces.

The **SELECT** statement has a special **WHERE** clause that can be used to check for columns with **NULL** values: the **IS NULL** clause. It looks like this:

► Input

```
SELECT prod_name
FROM products
WHERE prod_price IS NULL;
```

► Analysis

This statement returns a list of all products that have no price (that is, an empty **prod_price** field, not a price of 0), and because there are none, no data is returned.

The **customers** table, however, does contain columns with **NULL** values. The **cust_email** column contains **NULL** if a customer has no email address on file, and **IS NULL** can be used to identify these customers.

► Input

```
SELECT cust_name
FROM customers
WHERE cust_email IS NULL;
```

► Output

```
+-----+
| cust_name |
+-----+
| Mouse House |
| E Fudd     |
+-----+
```

Caution

NULL and Nonmatches You might expect that when you filter to select all rows that do not have a particular value, rows with a NULL will be returned. But they will not. Because of the special meaning of NULL, the database does not know whether they match, and so they are not returned when filtering for matches or when filtering for nonmatches.

When filtering data, make sure to verify that the rows with NULL in the filtered column are really present in the returned data.

Summary

In this chapter, you learned how to filter returned data by using the **SELECT** statement's **WHERE** clause. You learned how to test for equality, nonequality, greater than and less than, value ranges, and NULL values.

Challenges

1. Write a SQL statement to retrieve the product ID (`prod_id`) and name (`prod_name`) from the `Products` table and return only products with a price of 9.49.
2. Write a SQL statement to retrieve the product ID (`prod_id`) and name (`prod_name`) from the `Products` table and return only products with a price of 9 or more.
3. Now you'll test what you learned in Chapter 5 and this chapter. Write a SQL statement that retrieves the unique list of order numbers (`order_num`) from the `OrderItems` table for orders that contain 100 or more of any item.
4. Write a SQL statement that returns the product name (`prod_name`) and price (`prod_price`) from `Products` for all products priced between 3 and 6. Oh, and sort the results by price. (There are multiple solutions to this one, and we'll revisit it in the next chapter, but you can solve it using what you've learned thus far.)